

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №5
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Асинхронне програмування в телекомунікаційних системах: взаємодія з базою даних через SQLAlchemy.

Мета роботи: Ознайомитися з принципами асинхронного програмування в Python за допомогою модуля `asyncio`, навчитися працювати з асинхронними запитам до бази даних через SQLAlchemy, реалізувати систему збору та обробки даних про мережеві вузли.

Теоретичні відомості

Асинхронне програмування в Python

Асинхронний підхід дозволяє програмі виконувати декілька завдань одночасно, не блокуючи основний потік виконання. Основний інструментарій включає:

- Модуль `asyncio`: забезпечує роботу корутин та циклу подій (event loop).
- Корутина (`async def`): спеціальна функція, виконання якої можна призупинити та відновити пізніше.
- Ключове слово `await`: вказує програмі чекати на завершення асинхронної операції, дозволяючи в цей час виконувати інші завдання.
- `asyncio.run()`: стандартний спосіб запуску головної точки входу в асинхронну програму.

ORM SQLAlchemy та робота з БД

SQLAlchemy є бібліотекою, що реалізує ORM (Object-Relational Mapping), тобто відображення таблиць бази даних на класи Python. Ключові компоненти:

- `Engine`: об'єкт, який керує з'єднанням з базою даних.
- `Session`: інтерфейс для взаємодії з БД, через який проходять запити та транзакції.
- `Declarative Base`: базовий клас, від якого успадковуються моделі для автоматичного створення схем таблиць.
- Запит `select()`: використовується для вибору даних із таблиць у форматі об'єктів.

Асинхронна взаємодія з базою даних

Починаючи з версії 1.4, SQLAlchemy підтримує асинхронний режим роботи через `AsyncEngine` та `AsyncSession`.

- Для SQLite використовується драйвер `aiosqlite`, що дозволяє уникнути блокування програми під час дискових операцій.
- Метод `run_sync()`: необхідний для виконання синхронних операцій (наприклад, створення таблиць через `Base.metadata.create_all()`) всередині асинхронного контексту.
- Асинхронний контекстний менеджер `async with AsyncSessionLocal()` as `session` гарантує правильне відкриття та закриття сесії під час виконання запитів.

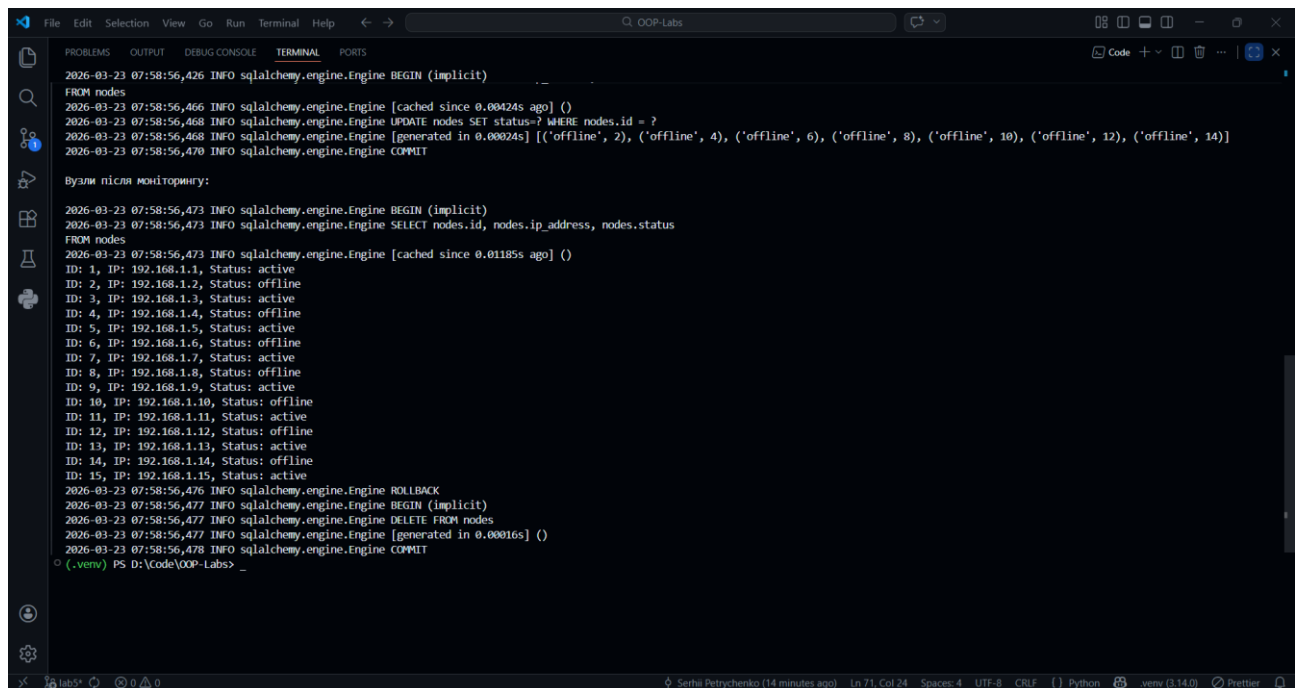
Результати виконаної роботи

```
File Edit Selection View Go Run Terminal Help Q. OOP-Labs
main.py X
main.py > get_nodes
1 """
2 1. Створити базу даних та таблицю nodes, яка зберігатиме інформацію про мережеві вузли (ID, IP-адресу, статус).
3 2. Реалізувати асинхронну функцію для отримання списку вузлів із бази.
4 3. Реалізувати асинхронну систему збору статусів вузлів (імітація мережевих запитів).
5 4. Зберігати отримані дані в базі за допомогою SQLAlchemy.
6 5. Продемонструвати та пояснити результати виконання програми (створення таблиці, додавання 10-15 вузлів, моніторинг з оновленням статусу, виведення списку вузлів до
7 та після оновлення т.д.).
8 """
9 import asyncio
10 from sqlalchemy import Column, Integer, String, text
11 from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
12 from sqlalchemy.orm import sessionmaker, declarative_base
13 from sqlalchemy.future import select
14
15 DATABASE_URL = "sqlite+aiosqlite:///network.db"
16 Base = declarative_base()
17
18 class Node(Base):
19     __tablename__ = 'nodes'
20     id = Column(Integer, primary_key = True)
21     ip_address = Column(String, unique = True, nullable = False)
22     status = Column(String, default = 'unknown')
23
24 engine = create_async_engine(DATABASE_URL, echo = True)
25 AsyncSessionLocal = sessionmaker(engine, class_ = AsyncSession, expire_on_commit = False)
26
27 async def create_tables():
28     async with engine.begin() as conn:
29         await conn.run_sync(Base.metadata.create_all)
30
31 async def add_nodes():
32     async with AsyncSessionLocal() as session:
33         nodes = [Node(ip_address = f'192.168.1.{i}', status = 'active') for i in range(1, 16)]
34         session.add_all(nodes)
35         await session.commit()
36
37 async def get_nodes():
```

```
File Edit Selection View Go Run Terminal Help Q. OOP-Labs
main.py X
main.py > ...
37
38 async def get_nodes():
39     async with AsyncSessionLocal() as session:
40         result = await session.execute(select(Node))
41         nodes = result.scalars().all()
42         for node in nodes:
43             print(f'ID: {node.id}, IP: {node.ip_address}, Status: {node.status}')
44
45 async def monitor_nodes():
46     async with AsyncSessionLocal() as session:
47         result = await session.execute(select(Node))
48         nodes = result.scalars().all()
49         for node in nodes:
50             node.status = 'offline' if int(node.ip_address.split('.')[1]) % 2 == 0 else 'active'
51             await session.commit()
52
53 async def reset_nodes():
54     async with AsyncSessionLocal() as session:
55         await session.execute(text('DELETE FROM nodes'))
56         await session.commit()
57
58 async def main():
59     await create_tables()
60     await add_nodes()
61
62     print('\nВузли перед моніторингом:\n')
63     await get_nodes()
64     await monitor_nodes()
65
66     print('\nВузли після моніторингу:\n')
67     await get_nodes()
68     await reset_nodes()
69
70     await engine.dispose()
71
72 if __name__ == '__main__':
73     asyncio.run(main())
```

```
File Edit Selection View Go Run Terminal Help Q. OOP-Labs
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
2026-03-23 07:58:56,426 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,426 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("nodes")
2026-03-23 07:58:56,427 INFO sqlalchemy.engine.Engine [raw sql] ()
2026-03-23 07:58:56,429 INFO sqlalchemy.engine.Engine COMMIT
2026-03-23 07:58:56,430 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,430 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,430 INFO sqlalchemy.engine.Engine [generated in 0.000905 (insertmanyvalues) 1/15 (ordered; batch not supported)] ('192.168.1.1', 'active')
2026-03-23 07:58:56,448 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,448 INFO sqlalchemy.engine.Engine [insertmanyvalues 2/15 (ordered; batch not supported)] ('192.168.1.2', 'active')
2026-03-23 07:58:56,449 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,449 INFO sqlalchemy.engine.Engine [insertmanyvalues 3/15 (ordered; batch not supported)] ('192.168.1.3', 'active')
2026-03-23 07:58:56,450 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,450 INFO sqlalchemy.engine.Engine [insertmanyvalues 4/15 (ordered; batch not supported)] ('192.168.1.4', 'active')
2026-03-23 07:58:56,450 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,450 INFO sqlalchemy.engine.Engine [insertmanyvalues 5/15 (ordered; batch not supported)] ('192.168.1.5', 'active')
2026-03-23 07:58:56,450 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,450 INFO sqlalchemy.engine.Engine [insertmanyvalues 6/15 (ordered; batch not supported)] ('192.168.1.6', 'active')
2026-03-23 07:58:56,451 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,451 INFO sqlalchemy.engine.Engine [insertmanyvalues 7/15 (ordered; batch not supported)] ('192.168.1.7', 'active')
2026-03-23 07:58:56,451 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,451 INFO sqlalchemy.engine.Engine [insertmanyvalues 8/15 (ordered; batch not supported)] ('192.168.1.8', 'active')
2026-03-23 07:58:56,452 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,452 INFO sqlalchemy.engine.Engine [insertmanyvalues 9/15 (ordered; batch not supported)] ('192.168.1.9', 'active')
2026-03-23 07:58:56,453 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,453 INFO sqlalchemy.engine.Engine [insertmanyvalues 10/15 (ordered; batch not supported)] ('192.168.1.10', 'active')
2026-03-23 07:58:56,453 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,454 INFO sqlalchemy.engine.Engine [insertmanyvalues 11/15 (ordered; batch not supported)] ('192.168.1.11', 'active')
2026-03-23 07:58:56,454 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,454 INFO sqlalchemy.engine.Engine [insertmanyvalues 12/15 (ordered; batch not supported)] ('192.168.1.12', 'active')
2026-03-23 07:58:56,455 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,455 INFO sqlalchemy.engine.Engine [insertmanyvalues 13/15 (ordered; batch not supported)] ('192.168.1.13', 'active')
2026-03-23 07:58:56,456 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,456 INFO sqlalchemy.engine.Engine [insertmanyvalues 14/15 (ordered; batch not supported)] ('192.168.1.14', 'active')
2026-03-23 07:58:56,457 INFO sqlalchemy.engine.Engine INSERT INTO nodes (ip_address, status) VALUES (?, ?) RETURNING id
2026-03-23 07:58:56,457 INFO sqlalchemy.engine.Engine [insertmanyvalues 15/15 (ordered; batch not supported)] ('192.168.1.15', 'active')
2026-03-23 07:58:56,458 INFO sqlalchemy.engine.Engine COMMIT
Вузли перед моніторингом:
2026-03-23 07:58:56,461 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,462 INFO sqlalchemy.engine.Engine SELECT nodes.id, nodes.ip_address, nodes.status
```

```
File Edit Selection View Go Run Terminal Help Q. OOP-Labs
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
2026-03-23 07:58:56,426 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,461 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,462 INFO sqlalchemy.engine.Engine SELECT nodes.id, nodes.ip_address, nodes.status
FROM nodes
2026-03-23 07:58:56,462 INFO sqlalchemy.engine.Engine [generated in 0.000306] ()
ID: 1, IP: 192.168.1.1, Status: active
ID: 2, IP: 192.168.1.2, Status: active
ID: 3, IP: 192.168.1.3, Status: active
ID: 4, IP: 192.168.1.4, Status: active
ID: 5, IP: 192.168.1.5, Status: active
ID: 6, IP: 192.168.1.6, Status: active
ID: 7, IP: 192.168.1.7, Status: active
ID: 8, IP: 192.168.1.8, Status: active
ID: 9, IP: 192.168.1.9, Status: active
ID: 10, IP: 192.168.1.10, Status: active
ID: 11, IP: 192.168.1.11, Status: active
ID: 12, IP: 192.168.1.12, Status: active
ID: 13, IP: 192.168.1.13, Status: active
ID: 14, IP: 192.168.1.14, Status: active
ID: 15, IP: 192.168.1.15, Status: active
2026-03-23 07:58:56,465 INFO sqlalchemy.engine.Engine ROLLBACK
2026-03-23 07:58:56,465 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,466 INFO sqlalchemy.engine.Engine SELECT nodes.id, nodes.ip_address, nodes.status
FROM nodes
2026-03-23 07:58:56,466 INFO sqlalchemy.engine.Engine [cached since 0.00424s ago] ()
2026-03-23 07:58:56,468 INFO sqlalchemy.engine.Engine UPDATE nodes SET status=? WHERE nodes.id = ?
2026-03-23 07:58:56,468 INFO sqlalchemy.engine.Engine [generated in 0.00024s] [('offline', 2), ('offline', 4), ('offline', 6), ('offline', 8), ('offline', 10), ('offline', 12), ('offline', 14)]
2026-03-23 07:58:56,470 INFO sqlalchemy.engine.Engine COMMIT
Вузли після моніторингу:
2026-03-23 07:58:56,473 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,473 INFO sqlalchemy.engine.Engine SELECT nodes.id, nodes.ip_address, nodes.status
FROM nodes
2026-03-23 07:58:56,473 INFO sqlalchemy.engine.Engine [cached since 0.01185s ago] ()
ID: 1, IP: 192.168.1.1, Status: active
ID: 2, IP: 192.168.1.2, Status: offline
ID: 3, IP: 192.168.1.3, Status: active
ID: 4, IP: 192.168.1.4, Status: offline
ID: 5, IP: 192.168.1.5, Status: active
```



```
2026-03-23 07:58:56,426 INFO sqlalchemy.engine.Engine BEGIN (implicit)
FROM nodes
2026-03-23 07:58:56,466 INFO sqlalchemy.engine.Engine [cached since 0.00424s ago] ()
2026-03-23 07:58:56,468 INFO sqlalchemy.engine.Engine UPDATE nodes SET status=? WHERE nodes.id = ?
2026-03-23 07:58:56,468 INFO sqlalchemy.engine.Engine [generated in 0.00024s] [('offline', 2), ('offline', 4), ('offline', 6), ('offline', 8), ('offline', 10), ('offline', 12), ('offline', 14)]
2026-03-23 07:58:56,470 INFO sqlalchemy.engine.Engine COMMIT

Вузли після моніторингу:
2026-03-23 07:58:56,473 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,473 INFO sqlalchemy.engine.Engine SELECT nodes.id, nodes.ip_address, nodes.status
FROM nodes
2026-03-23 07:58:56,473 INFO sqlalchemy.engine.Engine [cached since 0.01185s ago] ()
ID: 1, IP: 192.168.1.1, Status: active
ID: 2, IP: 192.168.1.2, Status: offline
ID: 3, IP: 192.168.1.3, Status: active
ID: 4, IP: 192.168.1.4, Status: offline
ID: 5, IP: 192.168.1.5, Status: active
ID: 6, IP: 192.168.1.6, Status: offline
ID: 7, IP: 192.168.1.7, Status: active
ID: 8, IP: 192.168.1.8, Status: offline
ID: 9, IP: 192.168.1.9, Status: active
ID: 10, IP: 192.168.1.10, Status: offline
ID: 11, IP: 192.168.1.11, Status: active
ID: 12, IP: 192.168.1.12, Status: offline
ID: 13, IP: 192.168.1.13, Status: active
ID: 14, IP: 192.168.1.14, Status: offline
ID: 15, IP: 192.168.1.15, Status: active
2026-03-23 07:58:56,476 INFO sqlalchemy.engine.Engine ROLLBACK
2026-03-23 07:58:56,477 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-23 07:58:56,477 INFO sqlalchemy.engine.Engine DELETE FROM nodes
2026-03-23 07:58:56,477 INFO sqlalchemy.engine.Engine [generated in 0.00016s] ()
2026-03-23 07:58:56,478 INFO sqlalchemy.engine.Engine COMMIT
(.venv) PS D:\Code\OOP-Labs>
```

Висновки

1. У ході роботи було розроблено асинхронну систему для керування мережевими вузлами. Зокрема, створено базу даних та таблицю nodes за допомогою SQLAlchemy. Було реалізовано програмну логіку для автоматичного додавання вузлів (10-15 одиниць), отримання їхнього списку та динамічного оновлення статусів (active/offline) без блокування основного потоку програми.
2. Цей підхід є основою для розробки сучасних систем моніторингу в телекомунікаціях. Наприклад, його можна застосувати для створення серверних панелей керування, які повинні одночасно опитувати тисячі пристроїв у мережі та миттєво записувати результати в базу даних, не змушуючи користувача чекати завершення кожного окремого запиту.
3. Виконання завдання дозволило практично опанувати механізми корутин та циклу подій через модуль asyncio. Було отримано досвід роботи з асинхронним ORM (SQLAlchemy 1.4+), що значно відрізняється від класичного синхронного підходу через використання AsyncSession та специфічних драйверів, таких як aiosqlite. Це дало розуміння того, як ефективно поєднувати об'єктно-орієнтоване програмування з асинхронними викликами до баз даних.
4. Переваги: висока продуктивність та масштабованість, оскільки програма не витрачає час на очікування відповіді від бази даних або мережі, виконуючи в цей момент інші завдання. Це дозволяє ефективніше використовувати ресурси сервера. Недоліки: підвищена складність написання та налагодження коду. Потрібно ретельно стежити за станом циклу подій (event loop) та використовувати лише асинхронні бібліотеки, оскільки будь-яка синхронна функція всередині корутини може заблокувати всю програму.